JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file.attribute

Interface BasicFileAttributes

All Known Subinterfaces:

`DosFileAttributes`, `PosixFileAttributes`

`public interface` **`BasicFileAttributes`**

Basic attributes associated with a file in a file system.

Basic file attributes are attributes that are common to many file systems and consist of mandatory and optional file attributes as defined by this interface.

Usage Example:

```
Path file = ...
BasicFileAttributes attrs = Files.readAttributes(file, BasicFileAttributes.clas
```

Since:

1.7

See Also:

`BasicFileAttributeView`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
`FileTime`	`creationTime()`	Returns the creation time.
`Object`	`fileKey()`	

		Returns an object that uniquely identifies the given file, or `null` if a file key is not available.
boolean	**isDirectory**()	Tells whether the file is a directory.
boolean	**isOther**()	Tells whether the file is something other than a regular file, directory, or symbolic link.
boolean	**isRegularFile**()	Tells whether the file is a regular file with opaque content.
boolean	**isSymbolicLink**()	Tells whether the file is a symbolic link.
FileTime	**lastAccessTime**()	Returns the time of last access.
FileTime	**lastModifiedTime**()	Returns the time of last modification.
long	**size**()	Returns the size of the file (in bytes).

Method Detail

lastModifiedTime

`FileTime lastModifiedTime()`

Returns the time of last modification.

If the file system implementation does not support a time stamp to indicate the time of last modification then this method returns an implementation specific default value, typically a `FileTime` representing the epoch (1970-01-01T00:00:00Z).

Returns:

a `FileTime` representing the time the file was last modified

lastAccessTime

`FileTime lastAccessTime()`

Returns the time of last access.

If the file system implementation does not support a time stamp to indicate the time of last access then this method returns an implementation specific default value, typically the `last-modified-time` or a `FileTime` representing the epoch (1970-01-01T00:00:00Z).

Returns:

`a FileTime representing the time of last access`

creationTime

`FileTime creationTime()`

Returns the creation time. The creation time is the time that the file was created.

If the file system implementation does not support a time stamp to indicate the time when the file was created then this method returns an implementation specific default value, typically the `last-modified-time` or a `FileTime` representing the epoch (1970-01-01T00:00:00Z).

Returns:

`a FileTime representing the time the file was created`

isRegularFile

`boolean isRegularFile()`

Tells whether the file is a regular file with opaque content.

Returns:

`true if the file is a regular file with opaque content`

isDirectory

`boolean isDirectory()`

Tells whether the file is a directory.

Returns:

`true if the file is a directory`

isSymbolicLink

`boolean isSymbolicLink()`

Tells whether the file is a symbolic link.

Returns:

`true if the file is a symbolic link`

isOther

```
boolean isOther()
```

Tells whether the file is something other than a regular file, directory, or symbolic link.

Returns:

`true if the file something other than a regular file, directory or symbolic link`

size

```
long size()
```

Returns the size of the file (in bytes). The size may differ from the actual size on the file system due to compression, support for sparse files, or other reasons. The size of files that are not `regular` files is implementation specific and therefore unspecified.

Returns:

`the file size, in bytes`

fileKey

```
Object fileKey()
```

Returns an object that uniquely identifies the given file, or `null` if a file key is not available. On some platforms or file systems it is possible to use an identifier, or a combination of identifiers to uniquely identify a file. Such identifiers are important for operations such as file tree traversal in file systems that support symbolic links or file systems that allow a file to be an entry in more than one directory. On UNIX file systems, for example, the *device ID* and *inode* are commonly used for such purposes.

The file key returned by this method can only be guaranteed to be unique if the file system and files remain static. Whether a file system re-uses identifiers after a file is deleted is implementation dependent and therefore unspecified.

File keys returned by this method can be compared for equality and are suitable for use in collections. If the file system and files remain static, and two files are the `same` with non-`null` file keys, then their file keys are equal.

Returns:

an object that uniquely identifies the given file, or null

See Also:

`Files.walkFileTree(java.nio.file.Path, java.util.Set<java.nio.file.FileVisitOption>, int, java.nio.file.FileVisitor<? super java.nio.file.Path>)`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.